Progress review

ML Pipeline Pro

From a raw CSV to a trained, monitored model, on our own machines, with an AI assistant that never acts without approval.

Team MJN

Project goal

Anyone can turn a CSV into a trained, monitored classifier, with Claude as the guide.

Prepare data

Upload, review the encoding, split 70/15/15.

Train & tune

PyTorch models chosen by validation score.

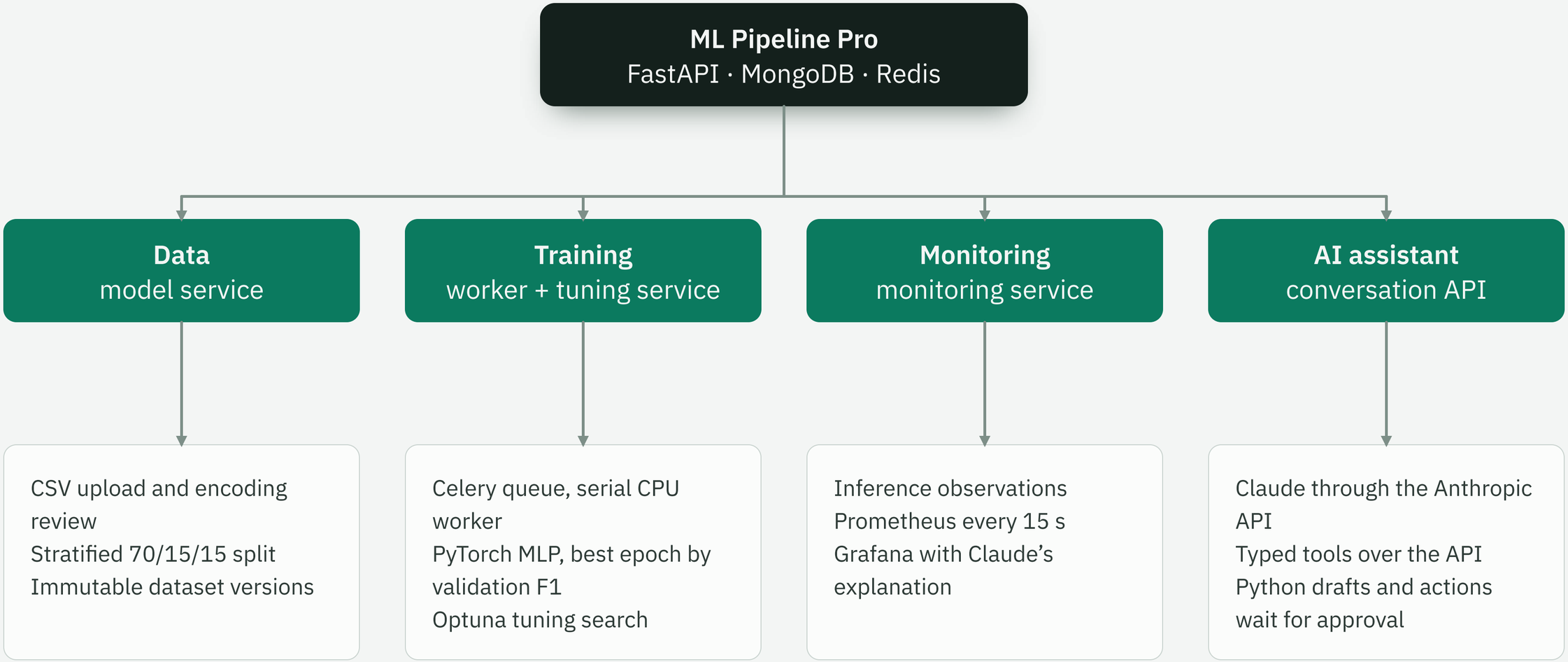
Monitor

Latency, accuracy and errors in Grafana.

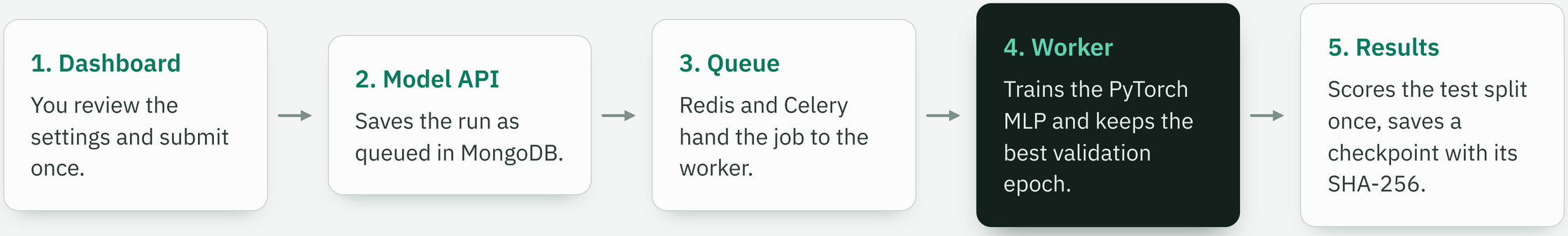
Assist

Claude explains results and proposes next steps.

One platform, four parts, each its own service



How one training run moves



Never twice

One idempotency key per submission, so a double click starts one run.

No test leakage

Scaling and model choice use training and validation rows only.

Repeatable

Fixed seed, deterministic PyTorch, one CPU thread.

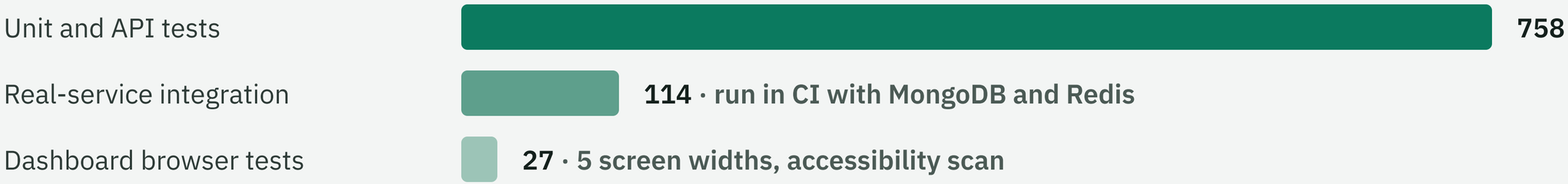
Progress we can measure

758
fast tests passing, 0 failing

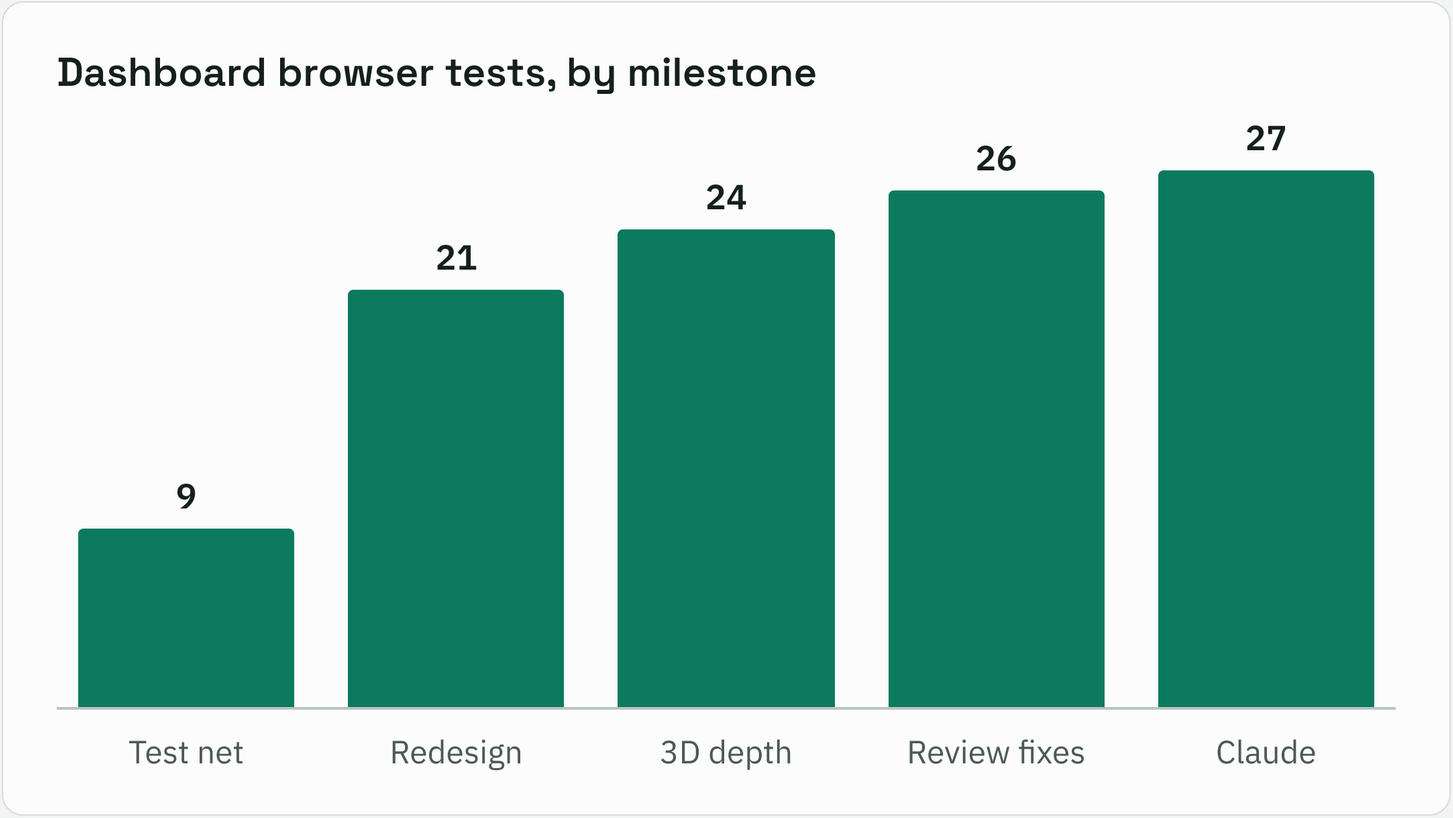
80%
code covered by those tests

14 / 14
Claude backend tests passing

Automated checks by suite



Measured progress over two days



740 → 758
Python tests, none failing

79% → 80%
code covered by tests

Same day
security finding found and fixed

Source: test runs at each commit, 19–20 Oct 2026, branches dashboard-redesign and claude-chat-backend.

Checkpoints: seven done, two open



7 of 9 done

1 in progress

1 open

Checkpoint	Status	Evidence
Core API, storage and queue	Done	Unit and integration tests
CPU training and Optuna tuning	Done	Real worker runs
Agent tools and Python drafts	Done	Typed tools, approvals, draft checks
Monitoring with Grafana	Done	20 inferences, visible in 4.1 s
Dashboard and CSV preparation	Done	Browser and worker runs, 19 Oct
Docker Compose and CI	Done	Hosted CI jobs passing
Claude API chat backend	Done	14 tests; key verified, live run next
Kubernetes deployment	In progress	MongoDB startup fix planned
Public deployment	Open	Vercel + Modal plan written

Prototype

A real training run, end to end, on a laptop

python examples/train_local.py

1.000
validation F1

~2 s
wall time on CPU

80/20/20
train / validation / test rows

2
files saved with SHA-256

Read it honestly

The example dataset is deliberately easy, so a perfect score proves the pipeline works, not that the model is strong.

Live demo: upload a CSV, review the encoding, train, and open the run in the redesigned dashboard.

Bottlenecks and risks

Laptop CPU inference is slow

Reading the first prompt dominates a chat turn; one comparison ran past the 270-second limit.

Plan: done — the agent now runs on Claude through the Anthropic API.

Kubernetes storage fails at startup

The dataset API returns 503: a database connection opens before MongoDB's replica set is ready.

Plan: replica-set discovery, then rerun the cluster smoke test.

Training can't run on Vercel

Vercel functions are short-lived and share no disk; training runs minutes and needs its files.

Plan: API and training on Modal with shared storage; Vercel serves the site.

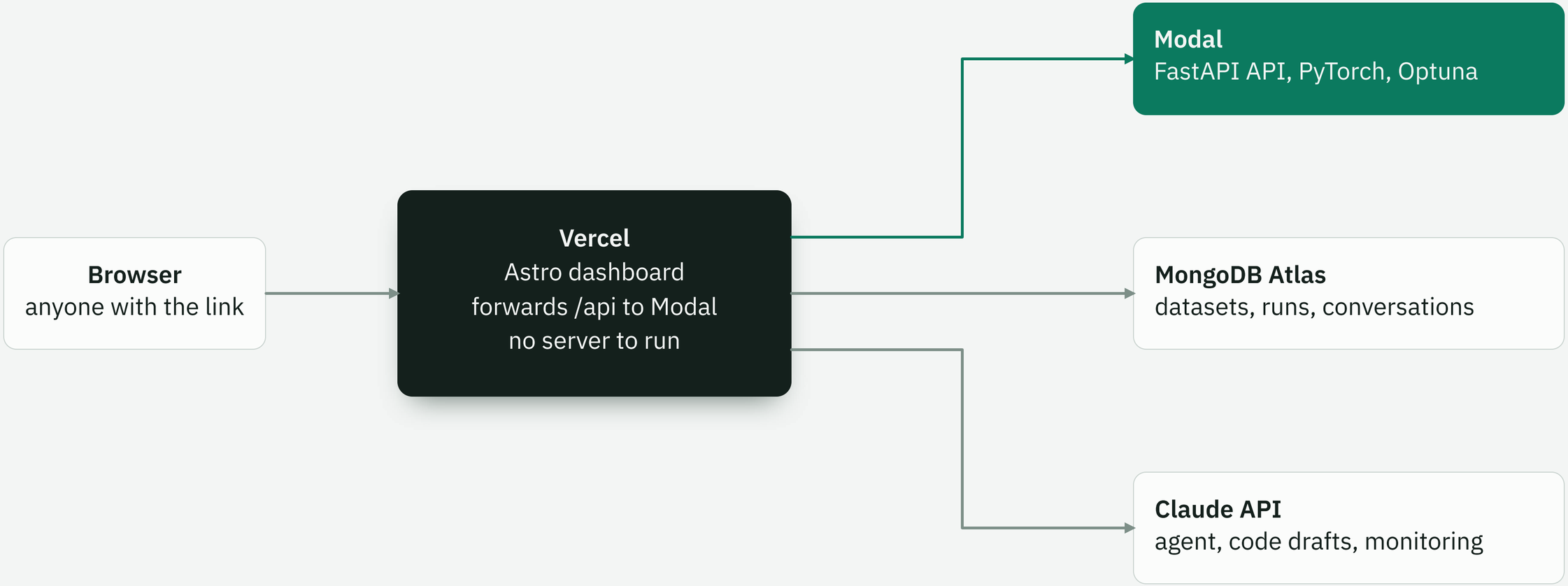
No independent reviewer yet

Concurrency and recovery code has only been reviewed by its own authors.

Plan: ask a peer team to review; disclose the gap until then.

Where it will run: nothing to keep switched on

Planned deployment. The Docker and Kubernetes setup stays in the repo for the course.



How we'll deploy: six steps

From our written deployment plan. Not started yet; local Docker and Kubernetes stay unchanged.

1 — Code

Jobs go to Modal

The API starts Modal train and tune functions instead of Redis and Celery; a Modal timer recovers lost jobs.

2 — Code + review

Locks move to MongoDB

File locks become MongoDB leases, so deleting a dataset still waits for training. Reviewed before launch.

3 — Code

Package both sides

Modal: API, training, one shared volume, secrets. Vercel: the dashboard, forwarding /api to Modal.

4 — Team

Create the accounts

A free MongoDB Atlas cluster, plus Modal and Vercel sign-ups; we log in ourselves.

5 — Team

Deploy

modal deploy, then vercel deploy. Keys live in Modal's secret store, never in the repo.

6 — Test

Prove it live

Upload, train, download and chat with Claude on the public link; record start-up time and cost.

Roadmap to the final delivery

1

Fix cluster storage

Replica-set discovery; Kubernetes smoke test green.

2

Merge the redesign

New dashboard into main once CI passes.

3

Claude live run

Real chat turns on Claude; confirm course fit.

4

Deploy

Vercel site, Modal API and training, Atlas data.

5

Final demo

Rehearse, review with a peer team, write the report.

Each step ends with a check we can show: a green test run, a merged pull request, a timed chat turn, a public link, a rehearsal under the time limit.



Seven checkpoints done. Two to go. A plan for both.

From a raw CSV to a monitored model, on our own machines.

Questions?